- Feature gaps between documentation and actual implementation
- Inconsistent behavior between different model versions from the same provider

Model capability mismatches

Many errors occur when attempting to use features that aren't supported:

- Image analysis capabilities might be mentioned in docs but not enabled for your tier.
- Audio transcription features may be region locked or require special access.
- Function calling might work differently than documented or be completely unavailable.
- Streaming responses might not be supported for certain model types.

Common run-time errors

Several types of errors typically manifest during normal operation:

- Empty responses when the model fails silently
- Max token limits enforced differently than documented
- Unclear error messages for content policy violations

So how do you mitigate those kinds of errors? Some of them, like region-locked features, can be verified by simply executing workflows and reviewing the responses for any relevant error messages. However, since there could be a variety of types of errors that can be thrown at any given interaction, it's important to at least implement a robust error-handling mechanism in key areas of your application logic and provide options for the user to recover or retry the request.

Implementing robust error handling with user-facing fallbacks

The simplest way to handle errors is to extend your existing `try-catch` block in server actions.jsx to catch potential errors. You can detect instances of an error either by using `instanceof` operators or by checking the error message contents.

> **NOTE** The source code for the examples in this chapter is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch08/chat-vercel-troubleshooting.

Project code: chat-vercel-troubleshooting

The code for the example project can be found in the ch08/chat-vercel-troubleshooting folder. You can use that example by running this command in the root folder of the project repository:

```
$ npm run dev -w ch08/chat-vercel-troubleshooting
```

You'll also need either the OpenAI API or the Google API keys to access their REST API. For details on creating API keys, refer to the appendix: